Semester 2 Fleet Maintenance Optimisation

Iteration 2 technical report and presentation preparation

David Long
University of Queensland and Northrop Grumman
12 October 2026

Main findings

In Semester 2, I extended the successful Iteration 1 priority-based selection approach with simulated annealing acceptance and a basin-hopping search, added prototype operational constraints, analysed search settings, and compared controlled starting schedules. The main recommendation is to retain priority-based selection with tuned neighbourhoods. The annealing experiments produced small smoothness gains but did not justify replacing the tuned baseline.

Worst-day availability 19 is impossible for the current forecast under the modelled maintenance rules. Six unavoidable D-check intervals overlap, limiting availability to 18. The priority method already reaches that bound. This establishes optimality for worst-day availability on this instance; it does not establish globally optimal smoothness.

Method or schedule	Worst day	Std deviation	Recorded time
As-loaded forecast	15	1.949767	Before search
Published priority baseline	18	1.413971	14.6 s
Tuned priority baseline	18	1.391860	115.6 s
Basin-hopping median of seeds 1 to 3	18	1.390417	306.3 s

The candidate improves standard deviation by 0.001444, approximately 0.10%, against the tuned baseline. It misses the required 0.005 gain and the typical runtime exceeds the five-minute limit. Two of three decision runs timed out. Mean availability remains 21.713010 aircraft because the check durations and check count are conserved.

How to use this report

The report follows the execution plan: annealing integration and results, the availability bound, operational constraints, hyperparameter analysis, and compact versus staggered versus as-loaded starts. It then records the remaining evidence gaps, provides a presentation narrative, and lists linked source files. The appendix contains all 74 recorded priority-method tuning trials.

Sources S1 to S7 and S10 to S16. Runtime values are individual recorded measurements or explicitly labelled medians, not fresh performance benchmarks.

1 Baseline and maintenance model

Iteration 1 established that prioritising checks on the worst availability days gives a practical deterministic search. The execution plan records an improvement from 15 to 18 aircraft and a runtime of approximately 15.4 seconds. The later local tuning record reproduces the published baseline at 18 aircraft, standard deviation 1.413971, and 14.6 seconds. I use those reproducible local scores for Semester 2 comparisons rather than combining rounded values from older slides. [S1, S2, S10]

Forecast and objective

Quantity	Value
Aircraft	24
Maintenance checks	278 comprising A 144 B 72 C 38 D 24
Base date	21 January 2026
Forecast horizon	Inclusive last day index 4480
Availability observations	4481 daily values for indices 0 through 4480
Total maintenance occupancy	10248 aircraft days
Average availability	21.713010 aircraft

The code scores schedules lexicographically: maximise worst-day availability first, minimise the standard deviation of daily availability second, and maximise mean availability third. Standard deviation uses the full daily series and the population convention in NumPy. A lower standard deviation means a smoother availability profile. A smoother schedule cannot outrank another schedule with better worst-day availability. [S2]

Maintenance rules

Check	Duration days	Interval days	Early start allowance days
A	14	365	42
B	28	730	70
C	84	1460	70
D	126	2920	70

The sequence is A, B, A, C, A, B, A, D. A check can start within its legal start window and must not overlap another check on the same aircraft. When a check moves, the aircraft's later checks are rebuilt using the interval reset rules. Larger checks reset the corresponding smaller-check clocks. The new start must lie inside the recalculated legal window, even when it moves later than its current start. A check cannot start after its due date, but it may finish after that date. [S2]

The baseline tries shifts of 28, 14, 7, and 3 days in either direction, in that order, with a 400-iteration cap. It searches the first priority level containing an improving neighbour and chooses the best improvement in that level. Availability is updated by removing the aircraft's old maintenance intervals and adding the rebuilt ones. [S2]

Precision correction: older notes call the horizon 4480 days. The current code treats 4480 as an inclusive index and computes 4481 daily observations. The quoted means and standard deviations use those 4481 observations.

2 Integration of simulated annealing

The main integration is in savn_priority_sa.py. It reuses the baseline's priority ordering, check rebuild, availability update, and scoring. The change is that feasible non-improving moves can enter the search. This matters because the original generate_neighbors function filters out every move that does not improve the score before an acceptance rule can see it. [S2, S3]

Function	Role
generate_feasible_neighbors	Proposes legal moves from the first priority level with feasible neighbours including non-improvements
accept_candidate	Applies temperature acceptance or the tightened rule
savn_optimise	Runs classic single-chain simulated annealing with cooling
greedy_polish	Runs deterministic improvement search with optional tabu and operational filters
basin_hopping_optimise	Combines warm start escape walk and tabu polish
start_vector and is_tabu_neighbor	Identify and reject the exact pre-escape schedule during polish

Classic simulated annealing

The classic SA function warm-starts from the published priority baseline by default, proposes a random feasible neighbour, applies the acceptance test, and multiplies temperature by cooling_rate after each iteration. The default temperature is 1.0 and the default cooling rate is 0.95. It stores a current schedule for exploration and a separate best-so-far schedule for reporting. This version contains the cooling mechanism described in the plan. [S3]

Acceptance rule

An improvement in the ranked score is accepted immediately. For a non-improving score, the loss is measured on the first worsened component: aircraft lost on the worst day, otherwise the increase in standard deviation, otherwise the decrease in average availability. The acceptance probability is the exponential of negative loss divided by temperature. At nonpositive temperature the code rejects non-improvements. At positive temperature an exact score tie has zero loss and is accepted. [S3]

The tightened option rejects a lower worst-day availability before the temperature test. The usual candidate permits a temporary reduction in worst-day availability, while retaining the best-so-far result. Every proposed move still has to satisfy maintenance rules and any active operational constraints. Temperature changes acceptance of legal schedules; it does not legalise a prohibited check movement. [S3, S4]

Difference from the execution plan

Slide 6 says that a lower worst day is always rejected. The later recorded decision allows that loss in the candidate and keeps the stricter rule as a fallback. Slide 4 describes cooling, which applies to classic SA. The measured basin-hopping candidate instead uses a fixed escape temperature throughout each run. Those distinctions should be stated in the upcoming presentation. [S1, S3, S8]

3 Basin hopping and earlier experiments

After classic SA failed to improve the published baseline, the search developed into basin-hopping with tabu polish. It first runs priority-based polish. Each escape cycle restarts from the protected best-so-far schedule, deliberately seeks a legal move with lower worst-day availability, then makes a plateau walk before polishing again. The exact pre-escape start vector is temporarily tabu during that polish so the search cannot immediately restore the same schedule. Only an improved polished result replaces best-so-far. [S3, S9]

The final decision settings use escape temperature 1.5, 15 escape cycles, 30 plateau moves, and up to 400 polish iterations. The escape loop permits one initial move plus up to 30 subsequent moves per cycle. There is no cooling of escape_temperature in basin_hopping_optimise. The settling stage is deterministic polish. [S3]

Earlier local results on the canonical workbook

Method and setting	Seed	Std deviation	Time seconds
Basin default T 1.5 cycles 15 plateau 30	99	1.406534	239.4
Classic SA T 1.5	99	1.413971	64.9
Classic SA T 2.0	7	1.413971	69.0
Classic SA T 1.5	42	1.413971	59.7
Basin T 2.0 plateau 30	7	1.402721	219.8
Basin T 2.0 plateau 30	42	1.402403	213.4
Basin T 1.5 plateau 30	123	1.404311	83.9
Basin T 2.0 plateau 40	99	1.403516	131.9

All eight records retain worst-day availability 18, mean 21.713010, and report rule compliance. The three classic SA runs return standard deviation 1.413971, equal to the published baseline, in approximately 60 to 69 seconds. The best exploratory basin run reaches 1.402403 in 213.4 seconds. That is a reduction of approximately 0.82% against the published baseline, but it is worse than the subsequently tuned priority baseline at 1.391860. It is also a best exploratory run rather than a three-seed median. [S12]

Regenerated cloud study

A separate 146-trial cloud study ran for 3183.7 seconds, about 53 minutes. Because the local workbook was absent remotely, the forecast was regenerated. Its warm start had worst-day availability 16, standard deviation 1.495149, and mean 21.690497. Its best result retained 16 and reduced standard deviation to 1.479924, about 1.02%. These trials demonstrate tuning effort on a different instance. They cannot establish performance on the canonical local forecast or prove availability 19 infeasible there. [S13, S14]

The early local verification and its Excel export have standard deviation 1.406534. The final decision median has 1.390417. These are different experiments and must not share the same result label. [S11, S17]

4 Final comparison and keep or drop decision

The latest decision experiment compares basin-hopping with the tuned priority baseline using the same half-step neighbourhoods. The deterministic baseline is measured once. The candidate and tightened fallback each use seeds 1, 2, and 3. The candidate's typical result is its median, with the range reported beside it. This avoids presenting the luckiest seed against a single deterministic result. [S4, S6, S7]

Run	Worst day	Std deviation	Seconds	Status
Tuned baseline	18	1.391860	115.6	Finished
Candidate seed 1	18	1.389453	285.5	Finished
Candidate seed 2	18	1.390417	318.4	Timed out
Candidate seed 3	18	1.390898	306.3	Timed out
Tightened seed 1	18	1.391860	56.4	Finished
Tightened seed 2	18	1.391860	63.5	Finished
Tightened seed 3	18	1.391860	106.3	Finished

Candidate median standard deviation is 1.390417. The range is 1.389453 to 1.390898, a spread of 0.001445. Median elapsed time is 306.3 seconds, approximately 2.65 times the recorded 115.6-second tuned baseline. The three candidate decision records together account for 910.2 seconds, about 15.2 minutes of measured execution. [S7]

Adoption requirement	Measured result	Pass
Gain of at least 0.005 in typical standard deviation	0.001444 or about 0.10%	No
Worst-day availability 18 on every seed	18 on seeds 1 2 and 3	Yes
Typical finished run within 300 seconds	306.3 seconds with two timeouts	No

Interpretation

The gain is real in the saved best-so-far scores, but it is small compared with the extra search effort. The recommendation is to retain the tuned priority baseline. The candidate's timed-out results are best-so-far snapshots at termination, not evidence that all planned cycles completed or that the search converged. [S7]

The timeout checks occur between iterations and escape moves rather than inside neighbour evaluation. A long evaluation can therefore overrun the nominal 300-second limit. The recorded overshoots range up to 721.3 seconds in the settings sweep. The measured elapsed time and timeout flag, rather than a claim of an exact five-minute cap, should be reported. [S3, S6, S7]

All tightened runs match the tuned baseline. In the present basin-hopping implementation the first escape move must lower worst-day availability, while tightened acceptance forbids that move. Thus the tightened option cannot start its escape phase and effectively returns the warm-start polish. Its shorter recorded times do not show a new successful annealing search. [S3]

5 Why worst day availability 19 is impossible

This result follows from mandatory maintenance occupancy. It is independent of how many random seeds or parameter combinations are tried. The current workbook contains 24 aircraft. Guaranteeing availability 19 would require no more than five aircraft in maintenance on every day. [S2, S5, S15]

Unavoidable interval of a D check

A first D check lasts 126 days and may start from 70 days before its due date through to its due date. The earliest placement finishes 56 days after due; the latest placement begins on due. Every legal placement therefore covers the 56-day interval from the due date to 56 days after it, with the end date excluded. In the report notation this is [due, due + 56).

The first D due date is fixed by the opening due date plus 7 times 365 days. Earlier A, B, or C checks do not reset the D clock. Moving earlier checks cannot remove the first D's unavoidable interval. The loaded first-D windows agree with those reconstructed from the opening due dates for all 24 aircraft. [S2, S5]

Aircraft	Unavoidable overlap
Aircraft 11	20 to 25 May 2033
Aircraft 14	20 to 25 May 2033
Aircraft 15	20 to 25 May 2033
Aircraft 19	20 to 25 May 2033
Aircraft 24	20 to 25 May 2033
Blue-8	20 to 25 May 2033

The six intervals coincide on day indices 2676 through 2681, corresponding to 20 through 25 May 2033. At least six aircraft must be down on those days, so availability can be no greater than 24 minus 6, which is 18. Consequently a worst-day availability of 19 is infeasible for this forecast and these rules. The baseline's achieved value of 18 matches the upper bound. [S5, S15]

What the bound does and does not establish

Total downtime is 10248 aircraft days. Across 4481 daily observations, average maintenance load is 2.286990 aircraft. That is below the five-aircraft-down allowance implied by target 19. The failure comes from concentrated unavoidable occupancy rather than insufficient average capacity.

The bound is conditional on the fleet, check durations, early-start allowances, and fixed opening due dates. Reaching 19 would require changing the instance or one of those assumptions. It cannot be achieved by tuning this optimiser alone. The model does not impose hangar-bay, crew, or parts capacity limits, so those are not the cause of this bound. [S2, S15]

The D-core overlap, dates, aircraft names, workbook counts, and loaded schedule compliance were independently recalculated from the current local workbook during preparation of this report.

6 Operational constraints and measured outcomes

The operational prototype adds three hard rules in a separate JSON configuration. A busy period requires a minimum number of aircraft available within an inclusive date range. A displacement limit bounds the absolute movement of each check from its as-loaded start, including changes caused by rebuilding later checks. A dependency group requires a stated number of its named aircraft to remain available on every day. [S4, S16]

The rules are dummy scenarios for research. The canonical maintenance workbook remains the input. Both feasible-neighbour generation and greedy polish can apply the operational filter. A hotter temperature never overrides an operational breach. The loaded schedule is checked first and a scenario that it already breaks is rejected without a repair search. [S3, S4, S6]

Scenario	Displacement days	Busy period minimum	Group minimum
Loose	70	15	2 of 4
Medium	42	15	2 of 4
Tight	21	16	3 of 4

All scenarios use 27 April to 26 May 2033 and 14 September to 13 October 2033. The two explicit dependency groups are Blue-6, Blue-7, Blue-8, Blue-9; and Aircraft 11, Aircraft 12, Aircraft 13, Aircraft 16. [S16]

Scenario	Worst day	Std deviation	Seconds	Outcome
Loose	18	1.506731	16.5	Zero final breaches
Medium	18	1.717945	3.3	Zero final breaches
Tight	Not searched	Not applicable	Not applicable	Loaded schedule rejected

These measured constrained outcomes are for tuned priority polish. Compared with the unconstrained value 1.391860, the operational limits leave less freedom to smooth the timeline while the loose and medium runs still achieve 18. Their shorter runtimes reflect an earlier stop within a smaller legal search space; they do not establish that added constraints make optimisation generally faster. [S7]

Meaning of the tight scenario result

Both busy periods reach only 15 on the loaded schedule against a requirement of 16. Each dependency group reaches only 2 available against a requirement of 3. These are four reported loaded-schedule breaches. They establish that the current starting schedule is inadmissible for the prototype. Because the runner performs no repair or alternative-start search, this is not a mathematical proof that no schedule can satisfy the tight scenario. [S4, S6, S7]

The execution-plan work asks for aircraft dependencies, while the written implementation spec asks for 3 of 4 even in loose and medium scenarios. The actual configuration relaxes those two scenarios to 2 of 4 so the loaded schedule is admissible. That adjustment should be documented explicitly. Candidate-method constrained comparison runs are not present in the latest result file, even though the filtering interface supports them. [S1, S6, S16, S20]

7 Priority method hyperparameter analysis

The local priority-method study contains 74 trials and records total elapsed time 3151.2 seconds, about 52.5 minutes. All 74 reported schedules pass the maintenance-rule compliance check. Forty-six reach worst-day availability 18, twenty-five reach 17, two reach 16, and one remains at 15. Average availability stays 21.713010 in every trial. [S10]

The study varies iteration budget, single shift sizes, variable-neighbourhood ladders, their order, richer offsets, selection policy, and refinement budgets. It reuses the baseline's scoring, rebuild, and neighbour code rather than replacing the maintenance model. The full trial register is in Appendix A. [S10, S18]

Setting	Worst day	Std deviation	Seconds
Published default	18	1.413971	14.6
Default with 160 iterations	17	1.468172	4.4
Default with 800 iterations	18	1.413971	15.0
Single shift size 7 days	18	1.470754	23.3
Small shifts first	17	1.590036	5.7
Add 42 day rung before default	18	1.404311	19.5
Dense weekly ladder	18	1.396502	82.7
Half-step neighbourhoods	18	1.391860	96.5
Best across all priority levels	18	1.415233	69.8

Recommended settings

Neighbourhood level	Offsets in days
First	-28 -14 +14 +28
Second	-14 -7 +7 +14
Third	-7 -3 +3 +7
Fourth	-3 -1 +1 +3

Keep the first-priority-level selection policy and the 400-iteration cap. The winner accepts 319 moves and reduces standard deviation by 0.022111, approximately 1.56%, against the published baseline. The default stops after 280 accepted moves. Increasing its cap to 800 reproduces the same result, while cutting it to 160 leaves worst-day availability at 17. [S10]

Large shifts first and layered neighbourhoods matter on this instance. Single small offsets and small-first ordering can consume the iteration budget before reaching 18. Collapsing every offset into one union is worse than the half-step ladder. Searching every priority level does not beat the current selection policy. These are measured instance-specific findings, not universal rules for all fleets. [S10]

Use the tuned neighbourhoods through greedy_polish or a caller that accepts shift_sets. The published baseline savn_optimise function itself does not expose a shift_sets argument and its original defaults remain the reproducibility reference. [S2, S3, S4]

8 Candidate method hyperparameter analysis

The latest candidate screening varies one setting at a time on the tuned neighbourhoods, with no operational constraints and screening seed 99. The reference settings are temperature 1.5, 15 escape cycles, 30 plateau moves, and 400 polish iterations. The eight planned variants run under a one-hour screening budget; a second seed is used only for the best screened variant when time remains. [S4, S6]

Changed setting	Seed	Std deviation	Seconds	Status
escape temperature 1.0	99	1.389293	142.8	Finished
escape temperature 2.0	99	1.391219	164.3	Finished
escape cycles 8	99	1.391219	308.3	Timed out
escape cycles 25	99	1.391860	311.6	Timed out
plateau moves 15	99	1.389614	191.3	Finished
plateau moves 45	99	1.391860	379.3	Timed out
polish iterations 200	99	1.396183	416.5	Timed out
polish iterations 800	99	1.391860	721.3	Timed out
escape temperature 1.0	1	1.391860	445.4	Timed out

Every saved screening result has worst-day availability 18 and mean 21.713010. Only three of the eight seed-99 variants finish before their per-run deadline: temperature 1.0, temperature 2.0, and 15 plateau moves. The second-seed repeat of temperature 1.0 times out as well. The nine elapsed records sum to 3080.8 seconds, approximately 51.3 minutes, excluding orchestration overhead. [S7]

What the screening supports

Temperature 1.0 is the best finished single-seed setting, at standard deviation 1.389293 in 142.8 seconds. Its gain against the tuned baseline is 0.002568, approximately 0.18%, and remains below the 0.005 adoption margin. The second seed returns only the baseline score when stopped at 445.4 seconds. Therefore this setting has no successful repeated-seed evidence of a reliable improvement. [S7]

Fifteen plateau moves also finish with a small improvement, while 45 plateau moves time out. Temperature 2.0 finishes with less improvement than 1.0 on the screening seed. The cycle-count variants and both polish-budget variants are truncated. Their saved scores should be read as what was reached before termination, rather than as fully converged sensitivity results. [S7]

The latest screening does not measure cooling-rate sensitivity, because the final candidate uses a fixed escape temperature. The execution plan’s cooling question belongs to classic SA or a future candidate with an explicit temperature schedule. It should not be described as a completed cooling study. [S1, S3, S7]

Overall tuning conclusion

The most dependable measured gain came from improving the priority method’s neighbourhoods. The early 146 cloud trials, eight local exploratory records, 74 priority trials, and latest decision and screening records show substantial experimentation. Their totals must stay separated by dataset, method, and purpose; adding them together would give a misleading impression of a single controlled study. [S7, S10, S12, S13]

9 Compact staggered and as loaded starts

This experiment addresses whether an initially staggered schedule differs from a compact overlapping schedule and the current as-loaded schedule. The implemented labels are bunched for compact, spread for staggered, and as-loaded for the current forecast. Each method runs from the same constructed start, without operational limits, so the measured start effect is not mixed with constraint effects. [S4, S6, S7]

Construction of the three starts

The opening check is pinned to its due date by the maintenance replay. The implemented variants therefore move check index 1, the next check on each aircraft. Bunched moves it toward the fleet median start, clamped inside its legal window. Spread selects the legal-window endpoint furthest from that median. Each aircraft's later checks are then rebuilt. The compact case aims to cluster checks across aircraft; it never permits overlapping checks on the same aircraft. [S2, S4]

Start	Before search std	Tuned result std	Candidate std seed 1
As-loaded	1.949767	1.391860	1.389453
Compact bunched	1.955482	1.404629	1.401288
Staggered spread	1.978174	1.417282	1.416022

All three starts have worst-day availability 15. After search every recorded result has 18. The as-loaded start is the smoothest both before and after optimisation. Neither constructing a compact start nor pushing the next checks toward more separated endpoints improves the final smoothness on this workbook. [S7]

Start	Tuned seconds	Candidate seconds	Candidate status
as-loaded	63.3	306.8	Timed out
bunched	47.8	169.5	Finished
spread	93.2	224.5	Finished

Bunching worsens the tuned result by 0.012768, about 0.92%, compared with as-loaded. Spreading worsens it by 0.025422, about 1.83%. Basin-hopping makes a small improvement over the tuned method within each start: 0.002407 for as-loaded, 0.003340 for compact, and 0.001260 for staggered. None reaches the 0.005 adoption margin. Only seed 1 was used for the candidate start comparison, so these are descriptive comparisons rather than repeated-seed estimates. [S7]

Limits of the start experiment

The variants preserve aircraft, check identities, durations, opening due dates, and rule compliance. They do not preserve every later due window: direct replay shows 23 changed latest_start values for bunched and 24 for spread. The experiment therefore changes initial legal placements and their subsequent maintenance resets. It is a controlled test of the implemented start constructor, rather than a redesign of opening maintenance ages or a globally maximally compact or maximally staggered fleet. [S2, S4, S5]

10 Workstream completion and evidence limits

All four execution-plan workstreams have an implementation and recorded evidence. The recommendation can be made now: keep the tuned priority method. The report also distinguishes what has been measured from what would require further experiments before a broader completion claim. [S1, S7]

Workstream	Recorded work	Evidence limit
SA integration	Classic SA and basin-hopping with best-so-far reporting	Candidate has fixed temperature and two decision runs time out
Operational constraints	Three dummy packs and legal loose and medium priority results	No candidate constrained result table and no tight-scenario repair
Hyperparameters	74 priority trials and candidate screening	Candidate repeat truncated and no final cooling-rate study
Initial stagger	Three starts compared using both methods	Next-check variants and only one candidate seed per start

Validation already represented in the project

The source includes tests for baseline maintenance replay, acceptance decisions, feasible neighbours, seeded behaviour, best-so-far protection, tabu polish, operational limits, start construction, and the numeric keep-or-drop rule. The priority tuning records explicitly mark all 74 schedules as rule compliant. The final study results separately record zero operational breaches for loose and medium. Recorded validity covers the modelled maintenance rules and dummy limits, rather than all requirements of a deployed fleet planning system. [S10, S19]

Remaining work before broader claims

For a complete comparison under operational limits, measure candidate runs on the loose and medium scenarios using the same neighbourhoods, start, runtime rule, and fixed seeds as priority polish. Keep the tight result labelled as an invalid loaded start unless a repair search or a separate feasibility analysis is performed.

For stronger evidence on initial stagger, define a quantitative measure of compactness or overlap before constructing more extreme starts. Repeat candidate runs across seeds if the aim is a robust stochastic comparison. If starting maintenance ages or opening due dates are changed, treat those as a different instance and recompute the forced-core bound.

For performance claims, reduce deadline overshoot and measure runs in a consistent environment. The deterministic tuned baseline has the same score but different recorded timings of 96.5, 115.6, and 63.3 seconds across study contexts. Thus the available timing ratio describes the decision records, not an isolated hardware-controlled speed benchmark.

Presentation corrections

Explain that the tested candidate can temporarily lower worst-day availability, uses fixed escape temperature, and returns best-so-far. Distinguish the proof that 19 is impossible from the rejection of the tight loaded start. Use the local JSON values instead of mixing old slide metrics or the regenerated cloud dataset. The source handover filename includes 30 October, but it is an existing report label; the current evidence snapshot is the local results available on 12 October 2026. [S1, S3, S7, S21]

11 Suggested presentation narrative

Iteration 1 foundation

“At the end of Iteration 1, priority-based selection gave us a practical method. It targeted the checks that affected our worst availability days, rebuilt each aircraft’s later checks when a move occurred, and updated the availability efficiently. On the local forecast it improved worst-day availability from 15 to 18 aircraft.”

What I added in Semester 2

“In Semester 2, I investigated whether allowing temporary worse moves could improve the result further. I retained the maintenance logic and the priority search. First I added classic simulated annealing acceptance. I then developed basin-hopping, where the search leaves a good schedule, walks through a different region, and uses priority polish to improve it again. The best schedule is protected throughout.”

What the experiments showed

“Classic simulated annealing did not improve the published baseline in the three recorded local runs. Basin-hopping did improve smoothness slightly, but the improvement became much smaller once I compared it fairly with a tuned priority method. The tuned priority result was 1.391860 standard deviation. The median basin-hopping result was 1.390417, about a 0.10 percent reduction. Two of the three runs timed out, and the median runtime exceeded five minutes. That does not meet the adoption rule.”

Why I stopped chasing availability 19

“I also established a bound on this forecast. A D check lasts 126 days but can only start up to 70 days early. That leaves 56 days when it must be in maintenance regardless of its legal start. Six aircraft share that unavoidable interval on 20 to 25 May 2033. With 24 aircraft, availability can only be 18 on those days. The baseline therefore reaches the best possible worst-day availability under these rules.”

Constraints and settings

“I added dummy busy periods, displacement limits, and dependency groups as hard rules. Loose and medium scenarios kept legal priority schedules at 18, although smoothing became harder. The tight pack was already invalid on the loaded plan, and this prototype does not repair it. In the 74-trial priority settings study, richer half-step neighbourhoods gave the strongest dependable gain, approximately 1.56 percent better smoothness than the original settings.”

Starting schedule and recommendation

“I compared compact, staggered, and as-loaded starts. The first check cannot move in this model, so these variants alter the next check and rebuild the later checks. All methods reached 18, but as-loaded gave the smoothest final result. This shows that the starting schedule matters, and simply spreading the next checks does not guarantee a better final plan. My recommendation is to retain priority-based selection with the tuned half-step neighbourhoods. The Semester 2 contribution is the measured comparison, the availability bound, the operational prototype, and the sensitivity evidence supporting that choice.”

This narrative describes the recorded experiments. Do not describe the candidate as cooling over time or the tight scenario as globally impossible without additional analysis.

12 Relevant implementation and data files

S1 Iteration 2 Execution Plan.pptx

Live 11-slide plan and notes. Defines the four workstreams, fairness intent, and the original acceptance wording.

S2 savn_priority_based_selection.py

Published deterministic baseline, workbook loader, scoring, priority ordering, and maintenance replay. Key functions: generate_neighbors and rebuild_aircraft_schedule.

S3 savn_priority_sa.py

Classic SA and the basin-hopping candidate. Key functions: accept_candidate, savn_optimise, greedy_polish, and basin_hopping_optimise.

S4 savn_iteration2.py

Tuned neighbourhoods, adoption rule, start constructor, screening schedule, and operational checks.

S5 heavy_maintenance_forecast.xlsx

Canonical local maintenance forecast. MxEvents supplies the aircraft, check starts, durations, and legal windows.

S6 .scratch/iteration-2/run_study.py

Runs decision, constraints, start schedules, and screening experiments. Resume logic skips already recorded runs.

S7 .scratch/iteration-2/study-results.json

Primary numeric record for the latest comparison, start schedules, constraints, and screening. Includes timeout flags.

S8 docs/adr/0001-temperature-may-lower-worst-day.md

Recorded acceptance decision permitting temporary worst-day losses in the candidate.

S9 docs/superpowers/specs/2026-09-17-basin-hop-tabu-plateau-design.md

Design history for protected-best escapes, plateau walks, and tabu polish.

S16 operational_constraints.json

Actual loose, medium, and tight busy-period, displacement, and dependency configurations.

13 Relevant experiments reports and exports

S10 .scratch/priority-based-selection/hyperparameter-results.json

All 74 local priority trials with settings, scores, accepted moves, runtime, and compliance.

S11 .scratch/priority-sa-hybrid/local-verification.json

Early canonical-workbook basin run: seed 99, standard deviation 1.406534, and 239.4 seconds.

S12 .scratch/priority-sa-hybrid/std-chase-trials.json

Eight exploratory local SA and basin records. Best basin standard deviation is 1.402403.

S13 .scratch/priority-sa-hybrid/hyperparameter-results.json

Compact summary of the 146-trial cloud study on the regenerated workbook.

S14 .scratch/priority-sa-hybrid/hyperparameter-report.md

Cloud-study interpretation, dataset mismatch, parameter trends, and search behaviour.

S15 docs/reports/2026-09-17-min19-infeasibility.md

Original mathematical forced-core argument and local workbook measurements.

S17 optimised_priority_sa_output.xlsx

Early exported original and optimised schedules and availability. Its summary score is 1.406534, not the final candidate median.

S18 .scratch/priority-based-selection/run_hyperparameter_study.py

Script behind the deterministic neighbourhood and selection study. The narrative report is alongside it as hyperparameter-report.md.

S19 test_savn_iteration2.py

Operational constraints, start schedules, tuned settings, decision logic, and screening tests. Companion suites: test_savn_priority_sa.py and test_savn_priority_based_selection.py.

S20 .scratch/iteration-2/spec.md

Detailed requirements and original implementation decisions. Some due-window and group-level wording differs from current behaviour.

S21 docs/reports/2026-10-30-iteration-2-handover.md

Earlier handover interpretation. Use the latest study JSON for numeric values and the present report for corrected implementation detail.

Appendix A All priority method tuning trials

All 74 records below are from the canonical local workbook. Every record reports maintenance-rule compliance and mean availability 21.713010. The trial name identifies its configuration; full offsets, selection policy, and iteration caps are in S10. “Stop” means neighbourhood exhaustion; “Cap” means the trial used its iteration budget. Times are recorded measurements.

No	Trial	Worst day	Std deviation	Seconds	End
1	baseline default	18	1.413971	14.6	Stop
2	iters 5	16	1.872941	0.0	Cap
3	iters 10	17	1.809552	0.0	Cap
4	iters 20	17	1.757626	0.1	Cap
5	iters 40	17	1.674928	0.3	Cap
6	iters 80	17	1.568984	1.0	Cap
7	iters 160	17	1.468172	4.4	Cap
8	iters 400	18	1.413971	14.9	Stop
9	iters 800	18	1.413971	15.0	Stop
10	single pm1	17	1.765355	3.3	Cap
11	single pm2	17	1.664235	4.2	Cap
12	single pm3	17	1.590036	8.2	Cap
13	single pm5	18	1.490649	13.6	Cap
14	single pm7	18	1.470754	23.3	Stop
15	single pm10	17	1.465586	5.3	Stop
16	single pm14	18	1.474391	3.8	Stop
17	single pm21	17	1.525571	2.4	Stop
18	single pm28	17	1.554409	1.5	Stop
19	single pm35	17	1.581875	1.3	Stop
20	single pm42	17	1.607762	0.9	Stop
21	single pm56	16	1.704382	0.3	Stop
22	single pm70	15	1.937597	0.0	Stop
23	vns default	18	1.413971	15.2	Stop
24	vns plus day	18	1.407168	20.3	Stop
25	vns plus wide	18	1.404311	19.5	Stop
26	vns plus month	18	1.407327	23.1	Stop
27	vns early window	18	1.409862	20.6	Stop
28	vns weekly	18	1.415076	19.6	Stop
29	vns fine	18	1.411760	14.8	Stop
30	vns coarse	17	1.524547	4.0	Stop
31	vns geometric	18	1.404470	22.4	Stop
32	vns geometric plus day	18	1.399695	94.9	Stop
33	vns three level	18	1.416179	58.2	Stop
34	vns two level 28 7	18	1.436985	25.8	Stop
35	vns two level 14 3	18	1.420113	37.7	Stop
36	vns monthish	18	1.407644	47.6	Stop
37	vns dense	18	1.396502	82.7	Stop

No	Trial	Worst day	Std deviation	Seconds	End
38	vns wide dense	18	1.402084	28.9	Stop
39	order small first default	17	1.590036	5.7	Cap
40	order small first weekly	17	1.765355	3.0	Cap
41	order interleaved	18	1.408437	13.6	Stop
42	order wide then day	17	1.488401	24.3	Cap
43	rich paired weeks	18	1.403516	41.6	Stop
44	rich one union	18	1.413656	42.4	Stop
45	rich dense union	18	1.406534	66.5	Stop
46	rich vns with halves	18	1.391860	96.5	Stop
47	asymmetric left heavy	18	1.419956	34.5	Stop
48	asymmetric right heavy	18	1.412866	38.3	Stop
49	select all levels default	18	1.415233	69.8	Stop
50	select first improving default	18	1.421370	45.5	Stop
51	refine rich vns with halves iters 80	18	1.562999	3.6	Cap
52	refine rich vns with halves iters 200	18	1.427636	20.0	Cap
53	refine rich vns with halves iters 800	18	1.391860	34.9	Stop
54	refine rich vns with halves all levels	18	1.403198	49.7	Stop
55	refine vns dense iters 80	17	1.568984	1.1	Cap
56	refine vns dense iters 200	17	1.442874	8.0	Cap
57	refine vns dense iters 800	18	1.396502	68.6	Stop
58	refine vns dense all levels	18	1.401448	72.5	Stop
59	refine vns geometric plus day iters 80	17	1.572394	4.9	Cap
60	refine vns geometric plus day iters 200	17	1.443957	30.9	Cap
61	refine vns geometric plus day iters 800	18	1.399695	987.3	Stop
62	refine vns geometric plus day all levels	17	1.398578	75.1	Stop
63	refine vns wide dense iters 80	17	1.567277	2.4	Cap
64	refine vns wide dense iters 200	18	1.448894	35.5	Cap
65	refine vns wide dense iters 800	18	1.402084	135.1	Stop
66	refine vns wide dense all levels	18	1.410337	157.9	Stop
67	refine rich paired weeks iters 80	18	1.562999	4.7	Cap
68	refine rich paired weeks iters 200	18	1.421370	27.2	Cap
69	refine rich paired weeks iters 800	18	1.403516	54.4	Stop
70	refine rich paired weeks all levels	18	1.415076	81.3	Stop
71	refine vns plus wide iters 80	17	1.574522	4.7	Cap
72	refine vns plus wide iters 200	17	1.438692	32.3	Cap
73	refine vns plus wide iters 800	18	1.404311	82.7	Stop
74	refine vns plus wide all levels	18	1.413971	40.8	Stop

Source S10. The documented outlier repeat runtime should not be treated as a stable property of its deterministic setting.